ING Fakultät Ingenieurwissenschaften

Fakultät Ingenieurwissenschaften

Echtzeitsysteme und mobile Robotik

Mobile Robotik mit ROS 2: ESP32-Eigenbau und TurtleBot 4

Wissenschaftliche Projektdokumentation

Autoren:	Leo Kilian, Clara Kratzsch
Matrikelnummern:	75205, 88474
Prüfer:	M.Sc. Moritz Thümmler

23. Oktober 2026

Inhaltsverzeichnis

Abbildungsverzeichnis **III**

Tabellenverzeichnis **IV**

Abkürzungsverzeichnis **V**

1 Einleitung und Zielsetzung **1**
1.1 Motivation und Projektüberblick 1
1.2 ROS 2: Konzeptuelle Grundlagen 1
1.3 Aufbau der Arbeitsumgebung 2

2 Erste Schritte mit ROS 2 und turtlesim **3**
2.1 Inbetriebnahme der turtlesim-Simulation 3
2.2 Erstellung eines eigenen ROS 2-Pakets 3
2.3 Entwicklung einer Steuerungs-Node für /cmd_vel 3

3 Programmierung des eigenen Roboters **5**
3.1 Hardwareübersicht und ESP32-Pinbelegung 5
3.2 Motoransteuerung über das ATtiny212-UART-Protokoll 5
3.3 Einbindung des ESP32 über micro-ROS 7
3.4 Umsetzung der Fahrfunktionalität über /cmd_vel 7
3.5 Sensorik und Anzeige 8
3.6 Nebenläufigkeit mit FreeRTOS 8
3.7 Sicherheitskette für Fahrbefehle 9
3.8 Eigenes Projekt: Erkundung mit Ultraschallkarte 9
3.9 Firmware-Update über WLAN (OTA) 11

4 Versuch: Arbeiten mit dem TurtleBot 4 **11**
4.1 Aufbau und Funktionsweise des TurtleBot 4 12
4.2 Vorbereitung: Simulation und manuelle Steuerung 12
4.3 Kartierung der Umgebung (SLAM) 13
4.4 Autonome Navigation auf der erstellten Karte 14
4.5 Umsetzung einer eigenen Beispielanwendung 15

5 Netzwerk- und Interprozesskommunikation in C **17**
5.1 Einstieg: Erste Schritte mit Sockets und Protokollentwurf 17
5.2 Socket-Kommunikation: Server, Client und gemeinsame Liste 17
5.3 Peer-to-Peer-Erweiterung der Socket-Anwendung 18
5.4 Interprozesskommunikation (IPC): Auswahl und Umsetzung 19
5.5 Server und Client mit Message Queues 19
5.6 Gleichberechtigte Kommunikation mit Shared Memory, Message Queues und Semaphoren 20

6 Auswertung und Fazit **22**
6.1 Zusammenfassung der Ergebnisse 22
6.2 Herausforderungen und Lösungsansätze 22
6.3 Reflexion und Verbesserungsvorschläge 23

Einsatz generativer KI **24**

Literaturverzeichnis **25**

Abbildungsverzeichnis

1 Systemübersicht: Host-Nodes, micro-ROS-Agent und Firmware des ESP32 (vereinfacht; Sensortopics `/range`, `/imu`, `/bumper`, `/wheel_cmd` laufen vom ESP32 über den Agenten zu den Host-Nodes) 5
2 Ultraschallkarte nach einer Erkundungsfahrt von 180 s (Fahrt E7, Schreibtisch mit begrenzten Rändern) 10
3 Gespeicherte Karte von Raum W213 (weiß: frei, schwarz: belegt, grau: unbekannt) 13
4 RViz während der Navigation am 18.08.2026: Ziel erreicht („reached“, Restdistanz 0,00 m, keine Recoveries) 14

Tabellenverzeichnis

1 Arbeitsumgebung 2
2 Abweichung nach einem Viereck in turtlesim abhängig von der Timerrate 4
3 Genutzte Komponenten und Anschlüsse 6
4 Experimentell ermitteltes Motorprotokoll 6
5 FreeRTOS-Tasks und Synchronisation in der Firmware 9
6 Erkundungsfahrten von je 180 s 11
7 Wesentliche Herausforderungen und Lösungen 23

Abkürzungsverzeichnis

AMCL	Adaptive Monte Carlo Localization
DDS	Data Distribution Service
GPIO	General Purpose Input/Output
I²C	Inter-Integrated Circuit
IPC	Interprocess Communication
IMU	Inertial Measurement Unit (inertiale Messeinheit)
Nav2	Navigation 2 (Navigationsframework für ROS 2)
OLED	Organic Light Emitting Diode
OTA	Over-the-Air (Firmware-Update über Funk)
QoS	Quality of Service
ROS 2	Robot Operating System 2
RViz	ROS Visualization (Visualisierungswerkzeug)
SLAM	Simultaneous Localization and Mapping
TCP	Transmission Control Protocol
TF	Transform (Koordinatentransformationen in ROS)
P2P	Peer-to-Peer
POSIX	Portable Operating System Interface
UART	Universal Asynchronous Receiver Transmitter
UDP	User Datagram Protocol
USB	Universal Serial Bus
WLAN	Wireless Local Area Network
XRCE	eXtremely Resource Constrained Environments (DDS-XRCE, Protokoll zwischen micro-ROS und Agent)

1 Einleitung und Zielsetzung

1.1 Motivation und Projektüberblick

Mobile Roboter verbinden Sensorik, Aktorik und Software, die unter Zeitvorgaben zusammenarbeiten müssen. Das Praktikum „Echtzeitsysteme und mobile Robotik“ vermittelt dafür den Umgang mit dem Robot Operating System 2 (ROS 2) in aufeinander aufbauenden Schritten: von der Simulation über einen selbst programmierten Roboter bis zu einem kommerziellen Robotersystem [1].

Der Beleg dokumentiert diese Schritte:

- Kapitel 2: ROS-2-Grundlagen mit turtlesim und ein eigenes ROS-2-Paket.
- Kapitel 3: Programmierung eines mobilen Roboters mit ESP32, Anbindung über micro-ROS, Sensorprojekt sowie Nebenläufigkeit mit FreeRTOS.
- Kapitel 4: Kartierung, Navigation und eine eigene Anwendung auf dem TurtleBot 4.
- Kapitel 5: Netzwerk- und Interprozesskommunikation in C.
- Kapitel 6: Auswertung und Fazit.

Die Kapitel 2 und 3 wurden von Leo Kilian bearbeitet, Kapitel 5 von Clara Kratzsch, Kapitel 4 gemeinsam. Der vollständige Quellcode liegt im öffentlichen Repository `https://github.com/clatoxa/EZMR`.

1.2 ROS 2: Konzeptuelle Grundlagen

ROS 2 ist eine Middleware mit Werkzeugen und Bibliotheken für Robotersoftware [2]. Eine Anwendung besteht aus *Nodes*, also eigenständigen Prozessen mit jeweils einer Aufgabe [3]. Nodes tauschen typisierte Nachrichten über benannte *Topics* nach dem Publish-Subscribe-Prinzip aus [4]. Sender und Empfänger kennen einander nicht; sie finden sich über das Topic. Die Übertragung übernimmt der Data Distribution Service (DDS), unter Humble standardmäßig Fast DDS. Nodes werden dabei ohne zentralen Server automatisch im Netzwerk gefunden. Über Quality-of-Service-Profile (QoS) lässt sich je Topic festlegen, ob Nachrichten zuverlässig zugestellt oder für später startende Empfänger vorgehalten werden. Neben Topics gibt es Services (Anfrage–Antwort) und Actions (lang laufende Aufträge mit Rückmeldung). Mehrere Nodes mit ihren Parametern startet ein *Launchfile* gemeinsam [5].

Mikrocontroller wie der ESP32 können kein vollständiges DDS ausführen. Hier setzt micro-ROS an: Auf dem Mikrocontroller läuft ein schlanker Client nach dem Protokoll DDS-XRCE, auf dem Host-PC ein *micro-ROS-Agent*. Der Agent vertritt den Mikrocontroller im ROS-2-Netzwerk und leitet dessen Nachrichten in beide Richtungen weiter [6]. Für das restliche System erscheint der ESP32 damit als gewöhnliche Node.

1.3 Aufbau der Arbeitsumgebung

ROS 2 Humble ist eine Version mit Langzeitunterstützung und wird als Binärpaket für Ubuntu 22.04 bereitgestellt [7]. Statt einer virtuellen Maschine wurde Ubuntu 22.04 nativ im Dual-Boot installiert. Gründe dafür waren der direkte USB-Zugriff auf den ESP32 und die direkte Erreichbarkeit des micro-ROS-Agents im Netzwerk. Tabelle 1 fasst die Umgebung zusammen.

Tabelle 1: Arbeitsumgebung

Komponente	Ausführung
Host-PC	AMD Ryzen 5 2600, Ubuntu 22.04.5 LTS (nativ, Dual-Boot)
ROS 2	Humble (`ros-humble-desktop`), Build-Werkzeug colcon (`ros-dev-tools`)
ROS-Workspace	`~/robotik_praktikum/ros2_ws`, Paket `shape_driver` (Python, rclpy)
micro-ROS-Agent	eigener Workspace `~/micro_ros_ws`, `micro_ros_setup` (Branch humble) [8]
ESP32-Entwicklung	VS Code mit PlatformIO, Arduino-Framework, `micro_ros_platformio` [9, 10]
Netzwerk	Host per Kabel, ESP32 per WLAN (2,4 GHz) am selben Router; Agent im UDP-Modus (Port 8888)
Versionsverwaltung	Git, GitHub-Repository mit eigenem Branch je Person

Die Host-Nodes sind in Python geschrieben, die ESP32-Firmware in C/C++ mit der micro-ROS-Client-Bibliothek rclc. Die Installation folgte der Praktikumsanleitung [1] und der offiziellen Dokumentation [7].

2 Erste Schritte mit ROS 2 und turtlesim

2.1 Inbetriebnahme der turtlesim-Simulation

turtlesim simuliert eine Schildkröte in einer 2D-Ebene und eignet sich zum Kennenlernen der ROS-2-Kommunikation [11]. Nach dem Start von `turtlesim_node` und `turtle_teleop_key` wurde der ROS-Graph mit den Kommandozeilenwerkzeugen `ros2 node` und `ros2 topic` untersucht [3, 4]. Die Simulation empfängt Bewegungsbefehle vom Typ `geometry_msgs/msg/Twist` auf `/turtle1/cmd_vel` und publiziert ihre Pose auf `/turtle1/pose`. Zusätzlich bietet sie Services (z. B. `/reset`, `/spawn`) und die Action `/turtle1/rotate_absolute`. Die Teleop-Node nutzt beide Wege: Pfeiltasten senden `Twist`-Nachrichten, Buchstabentasten Action-Ziele.

Zwei Beobachtungen waren für die weitere Arbeit wichtig:

- Ein Topic erscheint bereits, wenn nur ein Publisher *oder* ein Subscriber existiert. Seine Sichtbarkeit belegt also nicht, dass ein Empfänger läuft.
- Ein einzelner `Twist`-Befehl (`ros2 topic pub --once`) bewegt die Schildkröte nur kurz. turtlesim setzt die Geschwindigkeit auf null, wenn der letzte Befehl älter als 1 s ist. Eine Fahr-Node muss Befehle daher periodisch senden. Dasselbe Prinzip wurde später als Sicherheitsstopp in der ESP32-Firmware umgesetzt (Kapitel 3).

2.2 Erstellung eines eigenen ROS 2-Pakets

Im Workspace `~/robotik_praktikum/ros2_ws` wurde das Python-Paket `shape_driver` mit `ros2 pkg create --build-type ament_python` angelegt und mit `colcon build --symlink-install` gebaut [12, 13]. Damit das Paket gefunden wird, müssen zwei Umgebungen geladen sein: die ROS-Installation (*Underlay*, `/opt/ros/humble`) und der eigene Workspace (*Overlay*, `install/local_setup.bash`).

Ein Launchfile startet `turtlesim_node` und die eigene Node gemeinsam [5]. Alle Fahrwerte liegen in einer YAML-Parameterdatei. Code und Konfiguration sind dadurch getrennt: Für den realen Roboter (Kapitel 3) genügt eine zweite Parameterdatei, ohne den Code zu ändern.

2.3 Entwicklung einer Steuerungs-Node für /cmd_vel

Die Node `shape_driver_node` fährt ein regelmäßiges Vieleck als zeitgesteuerte Zustandsfolge FORWARD $\rightarrow$ TURN $\rightarrow \cdots \rightarrow$ DONE [14]. Ein Timer publiziert in jedem Zustand einen `Twist`-Befehl. Die Phasendauern ergeben sich aus der Geometrie:

$$t_{\text{gerade}} = \frac{l}{v}, \qquad t_{\text{dreh}} = \frac{2\pi/n}{\omega} \tag{1}$$

mit Seitenlänge l, Geschwindigkeit v, Eckenzahl n und Drehrate ω. Alle Größen sowie der Topicname und die Timerrate sind ROS-Parameter; $n = 3$ ergibt ein Dreieck, $n = 4$ ein Viereck. Nach `DONE` sendet die Node Nullbefehle und beendet sich.

Die Fahrt ist eine reine Steuerung ohne Positionsrückführung. Ihre Genauigkeit hängt deshalb von der Timerrate ab: Ein Phasenwechsel wird erst beim nächsten Timer-Takt erkannt, jede Phase verlängert sich also um bis zu eine Periode. Tabelle 2 zeigt den Effekt für ein Viereck ($l = 2$ Einheiten, $v = 1$ Einheit/s, $\omega = 0{,}5\,\mathrm{rad/s}$).

Tabelle 2: Abweichung nach einem Viereck in turtlesim abhängig von der Timerrate

	10 Hz	50 Hz
Drehphase (Soll 3,142 s)	3,20 s	3,16 s
Endwinkel (Soll 0°)	6,2°	2,1°
Abstand Endpose zu Startpose	0,13 Einheiten	0,04 Einheiten

Der Winkelfehler entspricht der erwarteten Verlängerung der vier Drehphasen. Eine höhere Rate verringert ihn hier etwa um den Faktor drei. Für die Simulation wurde daher 50 Hz gewählt.

3 Programmierung des eigenen Roboters

Abbildung 1 zeigt das Gesamtsystem. Der ESP32 erfasst Sensoren und steuert die Aktoren; Auswertung und Entscheidungen laufen gemäß Aufgabenstellung auf dem Host-PC [1]. Der Quellcode liegt im Repository unter `firmware/robot_base` (ESP32) und `ros2_ws/src/shape_driver` (Host).

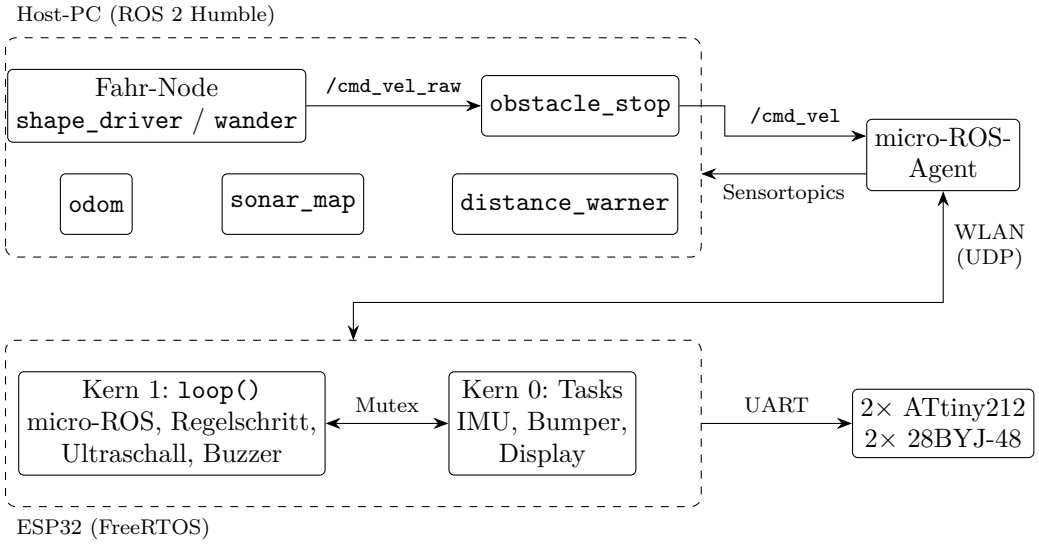


Abbildung 1: Systemübersicht: Host-Nodes, micro-ROS-Agent und Firmware des ESP32 (vereinfacht; Sensortopics `/range`, `/imu`, `/bumper`, `/wheel_cmd` laufen vom ESP32 über den Agenten zu den Host-Nodes)

3.1 Hardwareübersicht und ESP32-Pinbelegung

Der Roboter hat einen Differentialantrieb mit zwei Schrittmotoren und liegt vorn auf der Rahmenkante auf. Zentrale Einheit ist ein ESP32 (ESP32-D0WDQ6, zwei Kerne, 240 MHz) [15]. Tabelle 3 fasst die genutzten Komponenten zusammen. Die Pinbelegung stammt aus der Anleitung [1]; Abweichungen wurden mit einem eigenen Testprogramm (I^2C-Scan, Registerabfrage, Bumper-Abfrage) ermittelt.

GPIO 12 ist ein Strapping-Pin: Ist der linke Bumper beim Start eingesteckt, startet der ESP32 nicht [1, 17]. Daraus folgt die Regel, den Stecker vor jedem Einschalten, Reset und Upload abzuziehen. Flashen erfolgt über USB bei ausgeschaltetem Akku, der Fahrbetrieb über Akku ohne USB; der Akkuschalter dient als Not-Aus.

3.2 Motoransteuerung über das ATtiny212-UART-Protokoll

Jeder 28BYJ-48-Schrittmotor wird von einem eigenen ATtiny212 angesteuert, der die Schrittfolge erzeugt [1, 18]. Der ESP32 sendet nur Befehlspakete aus sechs Byte (`0xAA`, `id`, `mode`, `dir`, `param`). Baudrate, Motor-IDs, Richtungscodierung, Byte-Reihenfolge und die Einheit von `param` waren nicht dokumentiert. Sie wurden mit einer eigenen Testkonsole am aufgebockten Roboter ermittelt (Tabelle 4).

Tabelle 3: Genutzte Komponenten und Anschlüsse

Komponente	Anschluss	Befund
2× ATtiny212 + 28BYJ-48	UART, GPIO 17 (TX)	Protokoll experimentell ermittelt (Abschnitt 3.2)
HC-SR04 (Ultraschall)	GPIO 26 / 25	Höhe der Sensorachse ca. 3 cm, 10 cm vor der Radachse
IMU	I²C, 0x68	laut Anleitung GY-9250/MPU-9265; `WHO_AM_I` = 0x70 → MPU-6500 [16], vermutlich ohne Magnetometer
OLED 128×64	I²C, 0x3C	Controller SSD1306
Bumper links / rechts	GPIO 12 / 14	gedrückt oder abgezogen = LOW (fail-safe)
Piezo-Buzzer	GPIO 4	passiv, Tonerzeugung mit `tone()`
APDS-9960	I²C, 0x39	antwortet nicht, vermutlich nicht bestückt

Tabelle 4: Experimentell ermitteltes Motorprotokoll

Element	Ergebnis
Schnittstelle	9600 Baud, nur TX
`id`	1 = linkes, 2 = rechtes Rad
`dir`	spiegelbildlich: vorwärts links `dir 1`, rechts `dir 0`
`param`	16 Bit, Big-Endian
`mode 0`	feste Schrittzahl, 2048 Schritte je Radumdrehung; gepuffert, nicht abbrechbar
`mode 1`	Dauerbetrieb; `param` = Wartezeit je Schritt in ms (Abweichung < 1,5 %); neuer Befehl wirkt sofort; `param 0` = Stopp
Timeout	keiner im ATtiny (Motor lief > 20 s ohne neuen Befehl)

Ein Tippfehler zeigte die Bedeutung der Byte-Reihenfolge: 2026 wurde mit dem niederwertigen Byte zuerst gesendet und vom ATtiny als 59 911 Schritte gelesen. Die Konsole prüft seitdem `param` in beiden Lesarten gegen eine Obergrenze. Für den Fahrbetrieb kommt nur `mode 1` in Frage, da nur er sofortige Änderungen und Stopps erlaubt. Weil der ATtiny keinen Timeout hat, muss ein Stopp bei Verbindungsverlust vom ESP32 ausgehen.

3.3 Einbindung des ESP32 über micro-ROS

Die Firmware nutzt die Bibliothek `micro_ros_platformio` [10]. Gegenüber der Anleitung waren zwei Abweichungen nötig: `board_microros_distro = humble`, da die Bibliothek sonst eine neuere Distribution wählt, und `python3-rosdep` statt `python3-rosdep2` für den Agent-Workspace [8]. Plattform- und Bibliotheksversion sind in `platformio.ini` festgeschrieben, damit Builds reproduzierbar bleiben.

Der erste Test (Int32-Publisher aus der Anleitung) lief über USB. Mit `delay(100)` in `loop()` schwankte die Periode um ±90 ms. Ohne `delay()` lief ein Echo-Test (`/cmd_vel` → ESP32 → `/cmd_vel_echo`) mit 50 Hz bei einer Standardabweichung von 0,17 ms über 706 Intervalle. Für die Bodenfahrt wurde der Transport auf WLAN (UDP, Port 8888) umgestellt; `platformio.ini` enthält dafür die Umgebungen `serial` und `wifi` bei identischem Code. Die Zugangsdaten stehen in einer lokalen, von Git ausgeschlossenen Datei `secrets.h`.

3.4 Umsetzung der Fahrfunktionalität über /cmd_vel

Die Firmware rechnet jeden `Twist`-Befehl mit der Kinematik des Differentialantriebs in Radgeschwindigkeiten um:

$$v_{\mathrm{l,r}} = v \mp \omega \frac{L}{2}, \qquad \mathrm{param} = \frac{\pi d}{2048} \cdot \frac{1000}{v_{\mathrm{l,r}}} \tag{2}$$

mit Spurweite $L \approx 165\,\mathrm{mm}$, Raddurchmesser $d \approx 82\,\mathrm{mm}$ und `param` in ms je Schritt. Da `param` ganzzahlig ist, sind nur grobe Geschwindigkeitsstufen möglich (z. B. `param 2` = 62,9 mm/s, `param 4` = 31,5 mm/s). Zu große Befehle werden proportional begrenzt, damit die Kurvenform erhalten bleibt.

Für die Sicherheit sorgen drei Mechanismen: Ohne neuen `Twist` für 500 ms stoppen beide Räder; der aktuelle Befehl wird jede Sekunde wiederholt, da der UART keine Bestätigung kennt; beim Start und in der Fehlerschleife werden die Motoren gestoppt. Der Regelschritt läuft im Soll-Takt von 50 ms. Die erste Fassung (`last = now`) summierte Verspätungen auf und erreichte nur 18,6 Hz; mit `last += 50` liegt der Mittelwert bei 20,0 Hz.

Kalibrierfahrten ergaben 995 mm statt 1000 mm geradeaus und etwa 355° statt 360° beim Drehen. Da die groben `param`-Stufen eine Korrektur in der Firmware wirkungslos machen, wurden die tatsächlichen Geschwindigkeiten in der Parameterdatei `robot.yaml` hinterlegt; `shape_driver` berechnet damit die Phasendauern. Die Node aus Abschnitt 2.3 fährt so ohne Codeänderung ein Viereck (0,30 m Seitenlänge) auf

dem realen Roboter. Der Abstand zwischen End- und Startpunkt sank durch die Kalibrierung von etwa 35 mm auf 2–3 mm (Einzelfahrten).

Später wurde die Drehung zusätzlich über den Gierwinkel der IMU geregelt. Die Node stoppt dabei 4° vor dem Ziel, weil der Roboter durch die Übertragungskette im Mittel um etwa 3,5° nachdreht. Damit lag jede Ecke innerhalb von ±0,7° (direkt auf `/cmd_vel`) bzw. ±2,1° (über `obstacle_stop`). Unter gleichen Bedingungen waren kalibrierte Zeitsteuerung und IMU-Regelung gleichwertig.

3.5 Sensorik und Anzeige

Ultraschall. Der HC-SR04 misst alle 100 ms [19]. Die Echodauer wird per Interrupt aus den Flankenzeiten bestimmt statt mit `pulseIn()`, das bis zu 40 ms blockieren würde. Die Messwerte erscheinen als `sensor_msgs/Range` auf `/range`; Sonderfälle werden nach REP 117 als $+\infty$, $-\infty$ oder NaN gemeldet [20]. Gegen ein senkrechtes Buch betrug die Abweichung bis 50 cm höchstens 1 cm und bei 100 cm −1,5 cm.

IMU. Der Gierwinkel wird auf dem ESP32 mit 100 Hz aus der Drehrate integriert und als `sensor_msgs/Imu` auf `/imu` veröffentlicht. Der Nullpunkt des Gyroskops ($\approx -1{,}4\,°/s$) wird beim Start im Stillstand gemessen. Eine Drehung um 90° von Hand ergab 89,96–90,00°. Das Vorzeichen entspricht der ROS-Konvention [21].

Bumper, Buzzer und Display. Die Bumper melden auf `/bumper` (Bit 0 links, Bit 1 rechts). Gedrückt wird sofort erkannt, frei erst nach 100 ms stabilem Pegel, da beim Loslassen ein 55 ms langer Preller gemessen wurde. Der Buzzer erhält vom Host nur die Warnstufe auf `/buzzer` und erzeugt den Pieptakt lokal, damit WLAN-Schwankungen das Muster nicht verzerren. Das Display (Bibliothek U8g2 [22]) zeigt Abstand, Host-Zustand, Radbefehle und Gierwinkel. Die Übertragung eines Bildes dauert bei 400 kHz I^2C-Takt 31,9 ms.

3.6 Nebenläufigkeit mit FreeRTOS

Der ESP32 führt FreeRTOS auf zwei Kernen aus; `loop()` läuft auf Kern 1 [23, 24]. Alle micro-ROS-Aufrufe bleiben in `loop()`, da die Bibliothek nicht für mehrere Tasks ausgelegt ist. Zeitkritische oder blockierende Aufgaben wurden in eigene Tasks auf Kern 0 ausgelagert (Tabelle 5). Damit sind die drei Kernkonzepte der optionalen Aufgabe 2.6 umgesetzt: Task-Verwaltung, Ressourcenschutz und Ereignissteuerung [1].

Display und IMU teilen den I^2C-Bus. Das Display überträgt sein Bild deshalb in acht Seiten und gibt den Bus dazwischen frei; die IMU wartet so höchstens eine Seite statt 32 ms. Die Wirkung der Auslagerung zeigt ein Laufzeitvergleich über je 60 s: Mit dem Display als Task blieb der längste `loop()`-Durchlauf im Median praktisch unverändert (35,9 ms gegenüber 35,8 ms ohne Display). Direkt in `loop()` stieg er auf 66,3 ms, mit Spitzen von 574 ms. Das liegt über dem Motor-Timeout von 500 ms, der Roboter hätte während der Fahrt gestoppt.

Tabelle 5: FreeRTOS-Tasks und Synchronisation in der Firmware

Task (Kern 0)	Prio.	Auslöser und Synchronisation
`imuTask`	3	periodisch alle 10 ms (`vTaskDelayUntil`); I²C-Mutex, Mutex für IMU-Daten
`bumperTask`	2	Interrupt → binäre Semaphore; UART-Mutex für den Sofort-Stopp
`displayTask`	1	Task-Notification bei Zustandswechsel, sonst 4 Hz; Mutex für Anzeigedaten, I²C-Mutex je Bildseite

Der Sofort-Stopp über die Bumper-Task benötigte im Median 13,1 ms von der Flanke bis zum Ende der UART-Übertragung ($n = 10$). Davon entfallen rechnerisch 12,5 ms auf die zwei Pakete bei 9600 Baud; Interrupt, Semaphore und Aufwecken der Task dauern damit nur etwa 0,3 ms. Die Auswertung über den Host (Abschnitt 3.7) ist etwa zehnmal langsamer.

Eine verbleibende Grundlast mit `loop()`-Spitzen über 100 ms ging auf zuverlässig (*reliable*) gesendete Nachrichten zurück, die über WLAN auf Bestätigung warten. Mit *best effort* für die Sensorströme `/range`, `/imu` und `/wheel_cmd` sank das 90-%-Quantil des längsten Durchlaufs von 88,7 ms auf 23,9 ms. Die Host-Nodes abonnieren diese Topics passend mit dem QoS-Profil für Sensordaten, da ein *reliable*-Subscriber keine *best-effort*-Daten empfängt.

3.7 Sicherheitskette für Fahrbefehle

Auf `/cmd_vel` darf nur ein Publisher aktiv sein: Liefen zwei Sender gleichzeitig, wechselte der Roboter ständig zwischen beiden Befehlen. Die Node `obstacle_stop` sitzt deshalb als Filter zwischen Fahr-Node und Roboter (Abbildung 1). Fahr-Nodes publizieren per Parameter auf `/cmd_vel_raw`. Bei einem Hindernis wird nur die Vorwärtsfahrt gesperrt; Drehen und Rückwärtsfahren bleiben erlaubt, damit sich der Roboter befreien kann.

Der Filter arbeitet mit einem Median über drei Werte und einer Hysterese (Sperre unter 0,20 m, Freigabe über 0,25 m). Er sperrt auch, wenn 0,5 s lang keine Messwerte ankommen oder ein Bumper meldet. Seinen Zustand veröffentlicht er auf `/robot_state`. Vor einer Wand hielt der Roboter bei 19,4 cm. Die Reaktionszeit vom Eintreffen des Messwerts bis zum Stoppbefehl an die Motoren betrug im Mittel 159 ms ($n = 6$); davon entfallen etwa 100 ms auf den Medianfilter, der einen weiteren Messwert abwartet. Die Node `distance_warner` steuert parallel den Buzzer in vier Abstandsstufen. Eine Launch-Datei startet die gesamte Host-Seite mit einem Befehl [5].

3.8 Eigenes Projekt: Erkundung mit Ultraschallkarte

Als eigenes Projekt fährt der Roboter selbstständig umher, schätzt seine Position und erstellt aus den Ultraschallwerten eine Karte. Es ist ein vereinfachtes Gegenstück zur

Lidar-Kartierung des TurtleBot 4 (Kapitel 4). Erfassung auf dem ESP32 und Auswertung auf dem Host bleiben getrennt. Drei Nodes kommen hinzu, gestartet über `explore_launch.py`:

- `wander`: Zustandsautomat aus Vorwärtsfahrt, Zurücksetzen und IMU-geregelter 90°-Drehung. Nach einem Ultraschall-Hindernis misst er links und rechts und dreht zur freieren Seite; nach einem Bumper-Treffer dreht er von der getroffenen Seite weg.
- `odom`: Odometrie aus den tatsächlich gesendeten Radbefehlen (`/wheel_cmd`) und dem Gierwinkel; Ausgabe als `/odom` und Transformation `odom` → `base_link` [25].
- `sonar_map`: Belegungsraster mit 2 cm Zellen in Log-Odds-Darstellung [26]. Der Schallkegel wird durch sieben Strahlen angenähert (±7,5°, angenommen): Zellen davor gelten als frei, Zellen am Messwert als belegt [27].

Ein Viereckstest deckte eine Gyroskop-Drift von −0,31 °/s auf; der Endpunkt lag 4,8 cm neben dem Start. Vermutlich war der beim Einschalten gemessene Nullpunkt nach längerer Betriebszeit nicht mehr gültig (nicht nachgewiesen). Die Drift wird jetzt doppelt behandelt: Die Firmware misst den Nullpunkt auf Anfrage (`/imu_reset`) im Stillstand neu, und `odom` schätzt die Restdrift, solange beide Räder stehen. Mit der Driftschätzung lag der berechnete Endpunkt des Vierecks 2 mm neben dem Start; nach dem Neumessen des Nullpunkts betrug die geschätzte Restdrift nur noch 0,002–0,005 °/s.

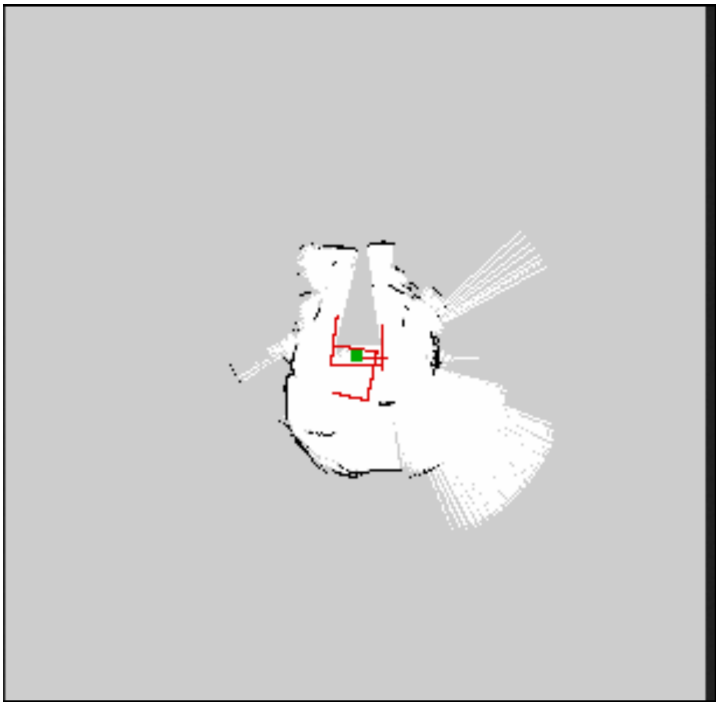


Abbildung 2: Ultraschallkarte nach einer Erkundungsfahrt von 180 s (Fahrt E7, Schreibtisch mit begrenzten Rändern)

Tabelle 6 vergleicht drei Erkundungsfahrten auf dem Schreibtisch. Mit Umschauen und Bumper-Seite (E7) traten die wenigsten Hindernisstopps auf, bei gleicher Strecke wie E5. Drehungen um 45° (E6) führten dagegen oft schräg auf Wände zu; schräg getroffene Flächen lenken den Schall vom Sensor weg und wurden spät erkannt (Deutung). Abbil-

dung 2 zeigt die Karte aus E7. Da jede Variante nur einmal gefahren wurde, sind die Unterschiede ein Hinweis, kein Nachweis.

Tabelle 6: Erkundungsfahrten von je 180 s

Fahrt	Strategie	Strecke	Hindernisse	Bumper
E5	±90°, Richtungswechsel nach 6–15 s	3,07 m	8	0
E6	±45°	2,39 m	24	5
E7	wie E5, zusätzlich Umschauen und Bumper-Seite	3,12 m	4	0*

* bei Geradeausfahrt; zwei weitere Kontakte während Drehungen

Scheinhindernisse durch Bodenecho. Auf dem Dielenboden sperrte `obstacle_stop` die Fahrt in zwei entgegengesetzten Richtungen, obwohl der Weg frei war. Ein Rundumscan zeigte Werte von 0,16–0,30 m nur quer zu den Dielenfugen. Der Messwert entsprach mit dem Zollstock dem Abstand zur zweiten Fuge, ein Blatt Papier auf dem Boden veränderte ihn, und ein leicht angehobener Sensor verschob ihn auf 0,45 m. Der untere Teil des Schallkegels trifft also den Boden, und Fugen quer zur Schallrichtung werfen ein messbares Echo zurück. Ursache ist die geringe Höhe des Sensors von etwa 3 cm, vermutlich verstärkt durch eine Neigung nach vorn. Die Sperren waren korrektes Verhalten des Filters auf ein falsches Signal. Eine niedrigere Schwelle hilft nicht, weil echte Hindernisse dann zu spät erkannt würden. Die Fahrten fanden deshalb auf fugenfreiem Untergrund statt. Als Abhilfe bleibt, den Sensor höher oder leicht nach oben geneigt zu montieren.

Die Kartenqualität begrenzen der breite Schallkegel (Bögen statt Konturen), die Odometrie ohne Radsensoren und ohne Kartenabgleich sowie das Bodenecho. Der Ultraschallsensor erkennt außerdem keine Tischkanten.

3.9 Firmware-Update über WLAN (OTA)

Damit für Updates nicht zwischen USB und Akku gewechselt werden muss, wurde ArduinoOTA nach Anleitung 4.1.4 eingebunden [1, 28]. Die PlatformIO-Umgebung `ota` lädt die Firmware per `espota` hoch [9]. Ziel-IP und Passwort kommen aus Umgebungsvariablen, damit keine Zugangsdaten im Repository stehen. Während des Uploads hängt `loop()` in `ArduinoOTA.handle()`; Timeout-Stopp und micro-ROS laufen dann nicht. Deshalb stoppt der Rückruf `onStart` zuerst die Motoren. Im Test drehte der Roboter auf der Stelle, als der Upload begann; die Räder stoppten, und das Update lief durch. Die Display-Task auf Kern 0 zeigt währenddessen den Fortschritt an. OTA ist auch aus der Fehlerschleife möglich, wenn kein Agent erreichbar ist.

4 Versuch: Arbeiten mit dem TurtleBot 4

Der Versuch nach Abschnitt 2.5 der Anleitung umfasst manuelle Steuerung, Kartierung, autonome Navigation und eine eigene Beispielanwendung auf dem TurtleBot 4 [1]. Er

fand am 18.08.2026 in Raum W213 statt, also nach den turtlesim-Grundlagen, aber *vor* der Arbeit am eigenen Roboter (Kapitel 3). Die Programme dieses Kapitels sind daher unabhängig vom eigenen Roboter entstanden. Verwendet wurde ein TurtleBot 4 Lite; als Host diente der Rechner `leobotsros` mit ROS 2 Humble, das auf Host und Roboter übereinstimmen muss [29].

4.1 Aufbau und Funktionsweise des TurtleBot 4

Der TurtleBot 4 besteht aus der Fahrbasis iRobot Create 3 mit Differentialantrieb und einem Raspberry Pi 4B [29, 30]. Die Create 3 stellt Antrieb, Batteriezustand, Stoß-, Absturz- und Infrarotsensoren sowie eine IMU als ROS-2-Schnittstellen bereit. Der Raspberry Pi führt die TurtleBot-4- und Sensor-Nodes aus und verbindet den Roboter per WLAN mit dem Netzwerk. Wichtigster zusätzlicher Sensor der Lite-Variante ist ein RPLIDAR-Laserscanner, der Abstandsdaten auf `/scan` (`sensor_msgs/msg/LaserScan`) liefert. Rechenintensive Pakete wie SLAM und Nav2 sowie die Visualisierung in RViz2 laufen auf dem Host.

Bewegt wird der Roboter wie in turtlesim über `geometry_msgs/msg/Twist` auf `/cmd_vel`; wirksam sind beim Differentialantrieb nur `linear.x` und `angular.z` [29].

4.2 Vorbereitung: Simulation und manuelle Steuerung

Zur Vorbereitung wurde die Simulation in Ignition Gazebo genutzt [31]. Da der physische und der simulierte Roboter dieselben Nodes und Schnittstellen verwenden, ließen sich Teleoperation, das Mitlesen von Topics und die Arbeit mit RViz ohne Anpassung übertragen. Am realen Roboter wurde dann mit `teleop_twist_keyboard` gefahren und `/cmd_vel` mitgelesen. Weitere Prüfungen (eigene Aufzeichnung, `bearbeitung.txt`): Der Laserscanner lieferte `/scan` mit im Mittel 7,4 Hz, `view_frames` zeigte die TF-Struktur mit Frames für Lidar, IMU, Räder sowie Stoß- und Absturzsensoren. In RViz waren unter anderem RobotModel, TF, LaserScan und Map aktiv.

Zusätzlich wurden zwei eigene Fahrprogramme auf dem TurtleBot 4 ausgeführt, die beide `Twist` auf `/cmd_vel` senden. Die nachträgliche Code-Analyse zeigt Abweichungen zwischen Absicht und Umsetzung:

- `turtlebot_shape.py` fährt zeitgesteuert ein Viereck oder eine Acht. Bei der Acht wird jeder Bogen mit 0,5 rad/s für 6,28 s gefahren, also nur ein Halbkreis; es entsteht eine S-Kurve statt einer Acht.
- `Kreisbahn.py` sendet $v\cos(\omega t)$ in `linear.x` und $v\sin(\omega t)$ in `linear.y`. In turtlesim ergibt das einen Kreis; ein Differentialantrieb kann `linear.y` aber nicht umsetzen, der Roboter pendelt daher vor und zurück. Für einen Kreis wären konstante Werte `linear.x` $= v$ und `angular.z` $= v/r$ nötig.

Ein in turtlesim funktionierendes Programm ist also nicht automatisch auf eine Plattform mit anderem Antriebsmodell übertragbar.

4.3 Kartierung der Umgebung (SLAM)

Beim SLAM (Simultaneous Localization and Mapping) baut der Roboter aus Laserscans und Odometrie eine Karte auf und verortet sich zugleich darin [32]. Kartiert wurde Raum W213 mit Tischen, Stühlen und zusätzlich verteilten Kartons. SLAM wurde ohne Zusatzargument gestartet, unter Humble also in der synchronen Variante von `slam_toolbox` [33, 34]. Der Roboter wurde per Tastatur durch den Raum gefahren, bis die Karte in RViz vollständig war; anschließend wurde sie über den Dienst von `slam_toolbox` gespeichert:

```
ros2 launch turtlebot4_navigation slam.launch.py
ros2 launch turtlebot4_viz view_robot.launch.py
ros2 service call /slam_toolbox/save_map slam_toolbox/srv/SaveMap "name:
  data: 'karte'"
```

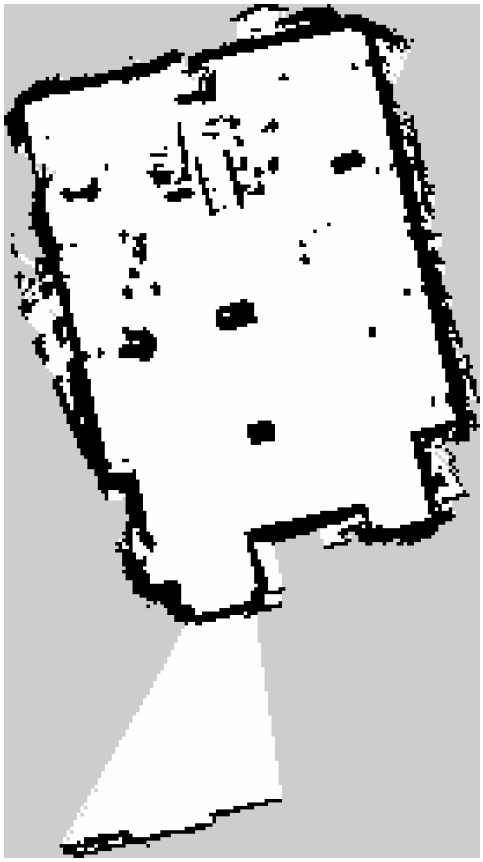


Abbildung 3: Gespeicherte Karte von Raum W213 (weiß: frei, schwarz: belegt, grau: unbekannt)

Die Karte (Abbildung 3, Repository: `leos_und_claras_karte.pgm/.yaml`) hat eine Auflösung von 0,05 m pro Pixel und deckt 6,85 m × 12,30 m ab. Die Wände sind als geschlossene Kontur erfasst, Tische, Stühle und Kartons als einzelne belegte Zellen. Ein keilförmiger freier Bereich unterhalb des Raums entstand vermutlich, weil der Laser durch eine offene Tür gemessen hat; für die Navigation im Raum ist er unerheblich. Drift oder doppelt erfasste Wände traten nicht auf.

4.4 Autonome Navigation auf der erstellten Karte

Für die Navigation wird auf der gespeicherten Karte lokalisiert, ohne sie zu verändern [32]. Lokalisierung und Nav2 [35] wurden in getrennten Terminals gestartet:

```
ros2 launch turtlebot4_navigation localization.launch.py
↪  map:=leos_und_claras_karte.yaml
ros2 launch turtlebot4_navigation nav2.launch.py
```

Nach dem Setzen der Anfangspose mit „2D Pose Estimate“ schätzt AMCL (Adaptive Monte Carlo Localization) die Position mit einem Partikelfilter [26]; `/amcl_pose` lieferte Posen im `map`-Frame mit Kovarianz. Mit „Nav2 Goal“ wird ein Ziel vorgegeben; Nav2 plant einen Pfad um die Hindernisse der Costmap und gibt Fahrbefehle auf `/cmd_vel` aus. Die aufgezeichnete Befehlsfolge zeigt eine Vorwärtsfahrt mit etwa 0,15 m/s, dann Abbremsen und eine Drehung auf der Stelle mit bis zu 0,37 rad/s, passend zum Ausrichten auf die Zielorientierung.

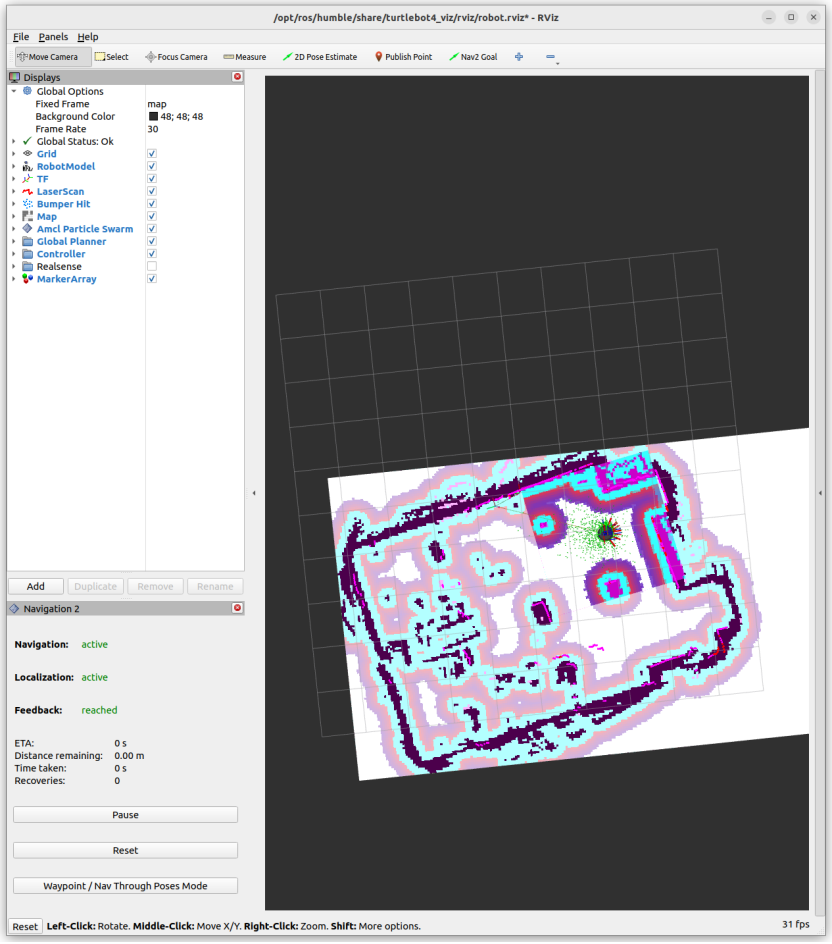


Abbildung 4: RViz während der Navigation am 18.08.2026: Ziel erreicht („reached“, Restdistanz 0,00 m, keine Recoveries)

Abbildung 4 zeigt eine erfolgreiche Zielanfahrt ohne Recovery-Verhalten; die AMCL-Partikel liegen eng um den Roboter. Nicht jede Zielvorgabe gelang im ersten Versuch,

die Fehlversuche wurden aber nicht protokolliert. Navigationsparameter wurden nicht verändert. Der Akku stand am 18.08.2026 bei nur 5 %. Damit ist der Ablauf aus Kartierung, Lokalisierung und Navigation nachgewiesen; für eine Aussage zur Zuverlässigkeit wären mehrere protokollierte Fahrten zu festen Zielen nötig.

4.5 Umsetzung einer eigenen Beispielanwendung

Als eigene Anwendung entstand der Hindernisausweicher `ausweicher.py`. Der Knoten abonniert `/scan`, wertet 40 Messwerte um die Mitte des `ranges`-Arrays aus (Quellcode 1) und publiziert auf `/cmd_vel`. Ein Zustandsautomat mit 10 Hz fährt geradeaus (`FAHREN`), bis der kleinste gültige Abstand unter 0,5 m fällt, und dreht dann für 1,5 s (`DREHEN`). Alle Werte sind ROS-Parameter. Der Knoten wurde ohne eigenes Paket direkt mit `python3 ausweicher.py` gestartet.

Quellcode 1: Auswertung des Frontbereichs im Scan-Callback (Auszug aus `ausweicher.py`)

```python
def scan_empfangen(self, msg):
    grenze = self.get_parameter('grenze').get_parameter_value().double_value
    anzahl = len(msg.ranges)
    mitte = anzahl // 2
    fenster = 20
    vorne = msg.ranges[mitte - fenster : mitte + fenster]
    gueltige = [d for d in vorne if d > 0.0 and math.isfinite(d)]
    if gueltige:
        naehester = min(gueltige)
        self.weg_frei = naehester > grenze
    else:
        self.weg_frei = False
```

Alle drei eigenen Knoten liefen auf dem TurtleBot 4; ein Video liegt im Repository (`VID_20260818_141102.mp4`). Das Ausweichverhalten passte jedoch nicht zuverlässig zur Lage der Hindernisse, und für eine Fehlersuche blieb im Versuch keine Zeit. Die Code-Analyse ergibt drei mögliche, *nicht* geprüfte Ursachen:

- **Richtung:** Der Code nimmt an, dass die Array-Mitte nach vorn zeigt. Das hängt aber von `angle_min` und von der Einbaulage des Scanners ab, die die Transformation von `base_link` nach `rplidar_link` beschreibt.
- **Fensterbreite:** 40 Messwerte entsprechen je nach Winkelauflösung nur einem schmalen Sektor; schräg vorn liegende Hindernisse werden übersehen.
- **QoS:** `/scan` wird mit dem Standardprofil (*reliable*) abonniert. Publiziert der Treiber mit *best effort*, kommen keine Daten an, und der Roboter dreht sich dauerhaft. Dieselbe Unverträglichkeit trat später am eigenen Roboter auf und wurde dort mit dem Profil für Sensordaten gelöst (Abschnitt 3.6).

Das Ziel wurde damit teilweise erreicht: Scan-Auswertung und Zustandsautomat sind umgesetzt, die Zuordnung der Messwerte zur Fahrtrichtung war nicht zuverlässig. Nächste Schritte wären ein Auswertefenster aus den Winkelangaben der Scan-Nachricht, das

QoS-Profil `qos_profile_sensor_data` und die Wahl der Drehrichtung zur freieren Seite, wie sie später in `wander` am eigenen Roboter umgesetzt wurde (Abschnitt 3.8).

5 Netzwerk- und Interprozesskommunikation in C

Ziel der folgenden Aufgaben ist die praktische Implementierung von Netzwerkkommunikation über Sockets sowie von Interprozesskommunikation (IPC) in C. Die Entwicklung erfolgte schrittweise von einem einfachen Echo-Server über ein eigenes Protokoll bis hin zu einer Peer-to-Peer-Anwendung.

5.1 Einstieg: Erste Schritte mit Sockets und Protokollentwurf

Als erster Einstieg in die Socket-Programmierung wurden die Beispiele aus der Man-Page `getaddrinfo(3)` als Ausgangspunkt verwendet. Diese implementieren einen einfachen Echo-Server sowie einen dazugehörigen Client auf Basis von UDP (`SOCK_DGRAM`). Der Server empfängt Datagramme mit `recvfrom` und sendet sie unverändert zurück; der Client übergibt Nachrichten als Kommandozeilenargumente.

Dabei zeigte sich, dass UDP für ein request-response-basiertes Protokoll mit mehreren aufeinanderfolgenden Kommandos pro Verbindung ungeeignet ist, da keine Verbindung aufgebaut wird und die Reihenfolge der Pakete nicht garantiert ist. Für die eigene Implementierung wurde daher TCP (`SOCK_STREAM`) gewählt, das eine zuverlässige, verbindungsorientierte Übertragung bietet.

Das Protokoll wurde als Klartextprotokoll entworfen, bei dem Kommandos und Parameter durch Leerzeichen getrennt werden:

```
<Kommando> <Parameter1> <Parameter2>
```

Die Auswertung erfolgt mit `sscanf`, was für ein einfaches Protokoll mit wenigen Kommandos ausreichend ist und ohne externe Bibliotheken auskommt.

5.2 Socket-Kommunikation: Server, Client und gemeinsame Liste

Server-Anwendung

Als Grundlage wurde zunächst eine einfache Server-Anwendung (`socket.c`) implementiert. Der Server bindet sich an Port 5071 und wartet in einer Endlosschleife auf eingehende TCP-Verbindungen. Für jede Verbindung wird eine innere Schleife gestartet, die Nachrichten des Clients entgegennimmt und auswertet.

Implementiert wurden die Kommandos `ADD`, `PRIME`, `LIST` und `CLEAR`. Fehlerhafte Anfragen werden ebenfalls in der Liste gespeichert, da so der vollständige Kommunikationsverlauf dokumentiert wird und Fehler im Nachhinein nachvollzogen werden können.

Client-Anwendungen

Parallel zum Server wurden zwei Client-Implementierungen entwickelt. Der erste Client (`client.c`) diente als einfacher Funktionstest: Er verbindet sich mit dem Server, sendet einmalig `ADD 5 10` und gibt die Antwort aus. Damit konnte der grundlegende Kommunikationsablauf schnell verifiziert werden.

Der zweite Client (`client1.c`) erweitert dies um eine interaktive Eingabeschleife. Der Benutzer kann beliebige Kommandos eingeben, die direkt an den Server gesendet werden. Bei `LIST` wird so lange empfangen, bis `LIST_END` eintrifft; alle anderen Kommandos erwarten genau eine Antwort. Mit `exit` wird die Verbindung sauber geschlossen. Für erste Tests wurde außerdem `netcat` verwendet, was den Vorteil hat, dass kein Client kompiliert werden muss und das Protokoll direkt auf der Kommandozeile getestet werden kann.

Gemeinsame Nachrichtenliste

Der Server verwaltet ein globales Array `messages[]` mit maximal 100 Einträgen. Jeder Eintrag speichert die ursprüngliche Anfrage und die zugehörige Antwort als Zeichenketten. Mit `LIST` kann ein Client die vollständige Liste abrufen. Die Einträge werden zeilenweise gesendet und mit `LIST_END` abgeschlossen, damit der Client weiß, wann die Übertragung beendet ist. Mit `CLEAR` kann die Liste geleert werden.

5.3 Peer-to-Peer-Erweiterung der Socket-Anwendung

Für die P2P-Erweiterung wurden Server und Client in einer einzigen Anwendung (`peer.c`) zusammengeführt. Jede Instanz kann gleichzeitig eingehende Verbindungen annehmen und selbst Verbindungen zu anderen Peers aufbauen.

Um beides gleichzeitig zu ermöglichen, ohne dass `accept` den Hauptthread blockiert, wurde auf POSIX-Threads (`pthread`) gesetzt. Der Hauptthread liest Benutzereingaben und sendet Kommandos an einen verbundenen Peer. Ein separater Thread übernimmt das Warten auf eingehende Verbindungen (`accept_connections`). Für jede neue Verbindung wird ein weiterer Thread gestartet (`handle_peer`), der die Kommunikation mit diesem Peer übernimmt. Diese Entscheidung wurde gegenüber `select` bevorzugt, da Threads konzeptionell klarer zu strukturieren sind und die Logik pro Verbindung unabhängig bleibt.

Da jeder Peer den anderen identifizieren muss, wurde das Protokoll um ein Port-Präfix erweitert:

```
PORT <AbsenderPort> <Kommando> [Parameter]
```

Der empfangende Peer extrahiert den Port des Senders und speichert ihn zusammen mit der Anfrage in der Liste, sodass in `LIST` nachvollziehbar ist, welcher Peer welches Kommando gesendet hat.

Synchronisation und bekannte Einschränkungen

Jeder Peer führt eine eigene lokale Liste. Diese wird nicht zwischen den Peers synchronisiert – ein `LIST`-Aufruf gibt ausschließlich die Einträge des befragten Peers zurück. Eine vollständige Replikation würde erfordern, dass jeder Schreibzugriff an alle verbundenen Peers weitergeleitet wird, was zusätzliche Konsistenz- und Reihenfolgeprobleme mit sich bringt und den Rahmen dieser Aufgabe übersteigt.

Da mehrere Threads grundsätzlich gleichzeitig auf `messages[]` schreiben könnten, wäre bei gleichzeitigen Zugriffen ein Mutex (`pthread_mutex_t`) erforderlich. Im praktischen Betrieb dieser Implementierung werden Kommandos jedoch sequenziell gesendet, sodass dieser Fall nicht auftritt.

5.4 Interprozesskommunikation (IPC): Auswahl und Umsetzung

Im Rahmen dieser Aufgabe wurden verschiedene IPC-Mechanismen untersucht. Zur Auswahl standen Shared Memory, Message Queues, Semaphoren sowie Pipes und Named Pipes. Für die grundlegende Kommunikation zwischen Server und Client wurde zunächst **Message Queues** gewählt, da sie konzeptionell dem Socket-basierten Protokoll aus Aufgabe 3 ähneln: Nachrichten bleiben als einzelne Einheiten erhalten, die Synchronisation ist im Kernel eingebaut, und ein request-response-Muster lässt sich direkt abbilden.

Für die erweiterte Peer-to-Peer-Variante wurden zwar Message Queues für den reinen Nachrichtenaustausch verwendet, für die Verwaltung der gemeinsamen Nachrichtenliste wurde jedoch **zusätzlich** Shared Memory gewählt. Dies war notwendig, da eine externe Datenstruktur gewünscht war, die unabhängig von den einzelnen Prozessen im Kernel existiert – die Liste soll nicht verschwinden, wenn Prozesse neu starten, sondern im Shared Memory weiterleben.

Zudem war es das Ziel, dass alle Peers direkt dieselbe zentrale Liste sehen, anstatt dass jeder Peer – wie bei vorherigen Ansätzen – seine eigene lokale Kopie verwaltet. Diese Anforderungen wurden in `peer_shm.c` umgesetzt, wobei der Zugriff auf den gemeinsamen Speicher **zusätzlich** durch Semaphoren synchronisiert wird, um Race Conditions bei gleichzeitigen Schreib- oder Leseoperationen auszuschließen.

5.5 Server und Client mit Message Queues

Die Grundlage bildet eine gemeinsame Message Queue mit dem festen Schlüssel `KEY = 1234`, die über `msgget` erstellt wird und unabhängig von den Prozessen im Kernel existiert. Anfragen vom Client werden mit `mtype = 1` in die Queue geschrieben; der Server liest gezielt Nachrichten dieses Typs mit `msgrcv`. Antworten werden mit `mtype = 2` zurückgeschickt, sodass Client und Server dieselbe Queue nutzen ohne sich gegenseitig Nachrichten wegzunehmen.

Der Server (`server_ipc.c`) wartet in einer Endlosschleife auf eingehende Nachrichten und wertet das Kommando im Textfeld der Nachricht aus. Implementiert sind die Kommandos `ADD`, `PRIME`, `LIST` und `CLEAR`. Da LIST-Antworten aus mehreren Einträgen be-

stehen können, wird für diese ein eigener Struct `ListEintrag` mit größerem Puffer verwendet, der alle Einträge einzeln in die Queue schreibt und mit `LIST_END` abschließt.

Der Client (`client_ipc.c`) öffnet dieselbe Queue ohne `IPC_CREAT`, da der Server sie zuerst erstellen muss. Nach dem Senden eines Kommandos wartet der Client auf die Antwort. Bei `LIST` empfängt er so lange bis `LIST_END` eintrifft.

Die Queue bleibt nach Beendigung der Programme erhalten und muss manuell mit `ipcrm` oder im Code mit `msgctl(msgid, IPC_RMID, NULL)` gelöscht werden.

Gemeinsame Liste via Message Queues

Der Server verwaltet ein globales Array `messages[]` mit maximal 100 Einträgen. Jeder Eintrag speichert die ursprüngliche Anfrage und die zugehörige Antwort. Fehlerhafte Anfragen werden ebenfalls gespeichert, da so der vollständige Kommunikationsverlauf nachvollziehbar bleibt.

Mit `LIST` kann der Client alle gespeicherten Einträge abrufen. Da LIST-Einträge die Kombination aus Anfrage und Antwort enthalten und damit größer sind als normale Nachrichten, wurde ein separater Struct `ListEintrag` mit einem Puffer von 450 Zeichen eingeführt. Dieser ist groß genug um zwei Felder von je 200 Zeichen plus festen Text aufzunehmen, ohne eine Kettenreaktion bei den Puffergrößen auszulösen.

Mit `CLEAR` wird die Liste geleert. Der Eintrag für `CLEAR` selbst wird nicht gespeichert, da dies die Liste direkt nach dem Leeren wieder befüllen würde.

5.6 Gleichberechtigte Kommunikation mit Shared Memory, Message Queues und Semaphoren

Für die Umsetzung der erweiterten Peer-Anwendung (`peer_shm.c`) werden die drei eingeführten IPC-Mechanismen kombiniert:

- **Message Queue** für den asynchronen Nachrichtenaustausch zwischen den Peers.
- **Shared Memory** für die zentrale, prozessübergreifende Nachrichtenliste.
- **Semaphor** zur Absicherung der kritischen Bereiche im Shared Memory.

Jeder Peer wird über die Kommandozeile mit einer eindeutigen numerischen ID gestartet:

```
./peer_shm <peer-id>
```

Anfragen werden gezielt mit `mtype = peer_id` des Empfängers in die gemeinsame Queue geschrieben. Antworten gehen mit `mtype = sender_id + 100` zurück, sodass der Empfangs-Thread (der ausschließlich auf `mtype = own_id` hört) Antworten nicht fälschlicherweise als neue Anfragen verarbeitet.

Der Benutzer steuert das Ziel seiner Nachrichten interaktiv im Terminal:

```
PEER <id>   -- Nachrichten an einen bestimmten Peer
ALL         -- Nachrichten an alle registrierten Peers
```

Beim Start registrieren sich Peers selbstständig im Shared Memory; beim Beenden oder bei unerwarteten Abbrüchen (z. B. durch `Strg+C`) sorgt ein implementierter Signal-Handler für `SIGINT` und `SIGTERM` für eine saubere Abmeldung und das Entfernen aus der Peer-Liste. Reagiert ein Ziel-Peer innerhalb eines definierten Timeouts nicht, bricht der Vorgang ab und das Ziel wird zurückgesetzt.

Bekannte Einschränkungen

Der Shared Memory wird beim ersten Programmstart im Kernel initialisiert. Wird das Programm ohne vorheriges Bereinigen neu gestartet, kann es zu Inkonsistenzen kommen. Die IPC-Ressourcen müssen nach der Nutzung manuell über die folgenden Befehle entfernt werden:

```
ipcrm -q <msgid>
ipcrm -m <shmid>
ipcrm -s <semid>
```

6 Auswertung und Fazit

6.1 Zusammenfassung der Ergebnisse

Alle Pflichtteile der Anleitung [1] wurden umgesetzt, dazu die optionalen Teile FreeRTOS und OTA:

- **ROS-2-Grundlagen** (Kapitel 2): Das eigene Paket `shape_driver` fährt in turtlesim ein Viereck; Fahrwerte und Topicname sind Parameter.
- **TurtleBot 4** (Kapitel 4): Eine Karte von Raum W213 wurde erstellt und autonom darauf navigiert. Der eigene Hindernisausweicher lief, wertete die Lidar-Daten aber nicht zuverlässig aus.
- **Eigener Roboter** (Kapitel 3): Das undokumentierte Motorprotokoll wurde experimentell ermittelt. Derselbe Knoten wie in turtlesim fährt nach einer Kalibrierung das Viereck mit 2–3 mm Positionsfehler, nur mit anderer Parameterdatei.
- **Sensorprojekt**: Ultraschall-Hinderniserkennung mit Sicherheitsfilter (Stopp 19,4 cm vor der Wand, Reaktionszeit 159 ms), Abstandswarnung, Bumper mit lokalem Sofort-Stopp (13 ms) und als eigenes Projekt eine Erkundungsfahrt mit Odometrie und Ultraschallkarte.
- **FreeRTOS und OTA**: Drei Tasks auf Kern 0 mit Mutexen, Semaphore und Notification entlasten die Regelschleife messbar; Firmware-Updates laufen über WLAN mit Motorstopp.
- **Socket- und Netzwerkkommunikation (Kapitel 5.1 - 5.3):** Entwurf eines textbasierten Klartext-Protokolls über TCP-Sockets zur Steuerung eines Server-Client-Modells sowie Erweiterung zu einer multi-threaded Peer-to-Peer-Anwendung.
- **Interprozesskommunikation und IPC-Modelle (Kapitel 5.4 - 5.6):** Umsetzung eines hybriden IPC-Ansatzes aus Message Queues für den Nachrichtenaustausch sowie Shared Memory und Semaphoren für eine persistente, synchronisierte Nachrichtenliste.

6.2 Herausforderungen und Lösungsansätze

Tabelle 7 fasst die wesentlichen technischen Herausforderungen zusammen.

Viele Fehler entstanden nicht im Code, sondern in der Arbeitsumgebung: falsche Dateiversionen nach dem Herunterladen, nicht neu gebaute Pakete, ein Agent, der nicht lief, oder eine falsche Startreihenfolge von Agent und ESP32. Geholfen haben eine feste Prüfroutine (`ros2 node list`, `ros2 topic info`, Kontrolle der installierten Dateien) und das Arbeitsprotokoll, in dem jeder Schritt mit Ergebnis festgehalten wurde.

Tabelle 7: Wesentliche Herausforderungen und Lösungen

Herausforderung	Lösung
Motorprotokoll nicht dokumentiert	systematische Versuche am aufgebockten Roboter, Testkonsole mit Plausibilitätsprüfung
ATtiny ohne Timeout	Timeout-Stopp (500 ms) und Befehlswiederholung im ESP32
mehrere Publisher auf `/cmd_vel`	ein Sicherheitsfilter als einziger Publisher, Fahr-Nodes auf `/cmd_vel_raw`
blockierende Displayübertragung (32 ms)	eigene Task auf Kern 0, I^2C-Mutex je Bildseite
lange `loop()`-Blockaden über WLAN	*best effort* für Sensorströme
Gyroskop-Drift nach längerer Betriebszeit	Nullpunkt neu messen, Driftschätzung im Stillstand
Scheinhindernisse durch Bodenecho	Ursache nachgewiesen; Fahrten auf fugenfreiem Untergrund
Strapping-Pin GPIO 12	Bumper vor Start und Upload abziehen

6.3 Reflexion und Verbesserungsvorschläge

Das stufenweise Vorgehen hat sich bewährt: Jede Funktion wurde zuerst einzeln und aufgebockt getestet, bevor der Roboter auf dem Boden fuhr. Die Trennung von Erfassung auf dem ESP32 und Auswertung auf dem Host machte die Sicherheitslogik austauschbar. Zeitkritische Reaktionen wie der Bumper-Stopp gehören dagegen lokal auf den Mikrocontroller; sie waren etwa zehnmal schneller als der Weg über den Host. Der Vergleich mit dem TurtleBot 4 zeigt den Abstand zu einem professionellen System: Lidar, Radencoder und SLAM liefern dort eine Karte mit klaren Konturen, während ein einzelner Ultraschallsensor mit reiner Befehlsodometrie nur eine grobe Umrisskarte ergibt.

Die meisten Messungen beruhen auf Einzelfahrten. Unterschiede zwischen Varianten, etwa bei den Erkundungsfahrten, sind daher Hinweise, keine Nachweise. Folgende Verbesserungen bieten sich an:

- Ultraschallsensor höher oder leicht nach oben geneigt montieren, um das Bodenecho zu vermeiden.
- Wiederholte Fahrten je Variante, um Streuungen anzugeben.
- Regelschritt in eine eigene periodische Task verlegen, um die verbleibende Verspätung von bis zu einem `loop()`-Durchlauf zu beseitigen.
- Automatische Wiederverbindung zum Agenten statt Reset per Taster.
- Beim TurtleBot-Ausweicher das Auswertefenster aus den Winkelangaben des Scans bestimmen und das QoS-Profil für Sensordaten verwenden.

Einsatz generativer KI

Bei der Bearbeitung des Projekts wurde generative KI als unterstützendes Werkzeug eingesetzt. Die Unterstützung umfasste insbesondere die Strukturierung von Arbeitsschritten, die Diskussion möglicher Fehlerursachen, Vorschläge für Codeänderungen sowie die strukturelle Ausarbeitung dieser Dokumentation. Sämtliche übernommenen Vorschläge wurden durch den Verfasser in der eigenen Entwicklungsumgebung geprüft, angepasst und verantwortet.

Literaturverzeichnis

[1] N. Beyer. *Echtzeitsysteme und mobile Robotik*. Praktikumsanleitung. Fakultät Ingenieurwissenschaften, Modulverantwortlicher: Moritz Thümmler. Hochschule für Technik, Wirtschaft und Kultur Leipzig, 8. Apr. 2026.

[2] S. Macenski u. a. „Robot Operating System 2: Design, Architecture, and Uses in the Wild“. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074.

[3] *Understanding nodes – ROS 2 Documentation: Humble*. Open Robotics. URL: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html (besucht am 12. 10. 2026).

[4] *Understanding topics – ROS 2 Documentation: Humble*. Open Robotics. URL: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html (besucht am 12. 10. 2026).

[5] *Integrating launch files into ROS 2 packages – ROS 2 Documentation: Humble*. Open Robotics. URL: https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Launch-system.html (besucht am 12. 10. 2026).

[6] *micro-ROS: ROS 2 for Microcontrollers*. micro-ROS. URL: https://micro.ros.org (besucht am 12. 10. 2026).

[7] *Ubuntu (deb packages) – ROS 2 Documentation: Humble*. Open Robotics. URL: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html (besucht am 12. 10. 2026).

[8] *micro_ros_setup*. GitHub-Repository. micro-ROS. URL: https://github.com/micro-ROS/micro_ros_setup (besucht am 12. 10. 2026).

[9] *Espressif 32 – PlatformIO Documentation*. PlatformIO Labs. URL: https://docs.platformio.org/en/latest/platforms/espressif32.html (besucht am 12. 10. 2026).

[10] *micro_ros_platformio*. GitHub-Repository. micro-ROS. URL: https://github.com/micro-ROS/micro_ros_platformio (besucht am 12. 10. 2026).

[11] *Using turtlesim, ros2, and rqt – ROS 2 Documentation: Humble*. Open Robotics. URL: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html (besucht am 12. 10. 2026).

[12] *Creating a workspace – ROS 2 Documentation: Humble*. Open Robotics. URL: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html (besucht am 12. 10. 2026).

[13] *Creating a package – ROS 2 Documentation: Humble*. Open Robotics. URL: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html (besucht am 12. 10. 2026).

[14] *Writing a simple publisher and subscriber (Python) – ROS 2 Documentation: Humble*. Open Robotics. URL: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html (besucht am 12. 10. 2026).

[15] *ESP32 Series Datasheet*. Espressif Systems. URL: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf (besucht am 12. 10. 2026).

[16] *MPU-6500 Product Specification*. PS-MPU-6500A-01. Version 1.0. InvenSense.

[17] R. Santos. *ESP32 Pinout Reference: Which GPIO pins should you use?* Random Nerd Tutorials. URL: https://randomnerdtutorials.com/esp32-pinout-reference-gpios/ (besucht am 12. 10. 2026).

[18] *Tutorial: 28BYJ-48*. URL: https://elektro.turanis.de/html/prj143/index.html (besucht am 12. 10. 2026).

[19] *Ultrasonic Ranging Module HC-SR04*. Datenblatt. ElecFreaks.

[20] *REP 117 – Informational Distance Measurements*. Open Robotics. URL: https://www.ros.org/reps/rep-0117.html (besucht am 12. 10. 2026).

[21] *REP 103 – Standard Units of Measure and Coordinate Conventions*. Open Robotics. URL: https://www.ros.org/reps/rep-0103.html (besucht am 12. 10. 2026).

[22] O. Kraus. *U8g2: Library for monochrome displays*. GitHub-Repository. URL: https://github.com/olikraus/u8g2 (besucht am 12. 10. 2026).

[23] *FreeRTOS (ESP-IDF) – ESP-IDF Programming Guide (ESP32)*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos.html (besucht am 12. 10. 2026).

[24] DigiKey. *Introduction to RTOS*. YouTube-Videoreihe. 2021. URL: https://youtube.com/playlist?list=PLEBQazB0HUyQ4hAPU1cJED6t3DU0h34bz (besucht am 12. 10. 2026).

[25] W. Meeussen. *REP 105 – Coordinate Frames for Mobile Platforms*. 2010. URL: https://www.ros.org/reps/rep-0105.html (besucht am 12. 10. 2026).

[26] S. Thrun, W. Burgard und D. Fox. *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005. ISBN: 978-0-262-20162-9.

[27] H. P. Moravec und A. Elfes. „High Resolution Maps from Wide Angle Sonar“. In: *Proceedings of the 1985 IEEE International Conference on Robotics and Automation*. 1985, S. 116–121. DOI: 10.1109/ROBOT.1985.1087316.

[28] *ArduinoOTA – Arduino core for the ESP32*. GitHub-Repository arduino-esp32, Bibliothek ArduinoOTA. Espressif Systems. URL: https://github.com/espressif/arduino-esp32/tree/master/libraries/ArduinoOTA (besucht am 12. 10. 2026).

[29] *TurtleBot 4 User Manual*. Clearpath Robotics. URL: https://turtlebot.github.io/turtlebot4-user-manual/ (besucht am 12. 10. 2026).

[30] *TurtleBot 4*. Clearpath Robotics. URL: https://clearpathrobotics.com/turtlebot-4/ (besucht am 12. 10. 2026).

[31] *TurtleBot 4 Simulator – TurtleBot 4 User Manual*. Clearpath Robotics. URL: https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_simulator.html (besucht am 12. 10. 2026).

[32] *Navigation – TurtleBot 4 User Manual*. Clearpath Robotics. URL: https://turtlebot.github.io/turtlebot4-user-manual/tutorials/navigation.html (besucht am 12. 10. 2026).

[33] *Generating a Map – TurtleBot 4 User Manual*. Clearpath Robotics. URL: https://turtlebot.github.io/turtlebot4-user-manual/tutorials/generate_map.html (besucht am 12. 10. 2026).

[34] S. Macenski und I. Jambrecic. „SLAM Toolbox: SLAM for the dynamic world“. In: *Journal of Open Source Software* 6.61 (2021), S. 2783. DOI: 10.21105/joss.02783.

[35] S. Macenski u. a. „The Marathon 2: A Navigation System“. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020. DOI: 10.1109/IROS45743.2020.9341207.